

.....

Author: Dev Kinger

The focus is on creating a few interactive diagrams for analysis

.....

Imports

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from matplotlib.gridspec import GridSpec

Plot styling (kept consistent across all figures)

plt.style.use("seaborn-v0_8-darkgrid")

sns.set_palette("husl")

plt.rcParams["figure.dpi"] = 300

plt.rcParams["savefig.dpi"] = 300

plt.rcParams["font.family"] = "sans-serif"

plt.rcParams["font.size"] = 11

plt.rcParams["axes.labelsize"] = 12

plt.rcParams["axes.titlesize"] = 14

plt.rcParams["xtick.labelsize"] = 10

plt.rcParams["ytick.labelsize"] = 10

plt.rcParams["legend.fontsize"] = 10

```

# -----
# Colour palette (chosen for clarity in presentations)
# -----

COLORS = {
    "positive": "#2ecc71",
    "negative": "#e74c3c",
    "neutral": "#95a5a6",
    "primary": "#3498db",
    "secondary": "#9b59b6",
    "accent": "#f39c12",
    "risk_high": "#c0392b",
    "risk_low": "#27ae60",
}

# =====
# Load and prepare data
# =====

print("Loading data from Book1.xlsx...")

industry_raw = pd.read_excel("Book1.xlsx", sheet_name="Industry")
company_raw = pd.read_excel("Book1.xlsx", sheet_name="Company")

# First row contains year labels, so skip it
industry = industry_raw.iloc[1:].copy()
company = company_raw.iloc[1:].copy()

industry.columns = [
    "Section", "Section_Code",
    "Neg_2024", "Neg_2025", "Change_Sentiment",
    "Pos_2024", "Pos_2025",
    "Neg_Val_2024", "Neg_Val_2025",

```

```

        "Neu_2024", "Neu_2025"
    ]

company.columns = [
    "Section", "Section_Code",
    "Neg_2023", "Neg_2024", "Change_Sentiment",
    "Pos_2023", "Pos_2024",
    "Neg_Val_2023", "Neg_Val_2024",
    "Neu_2023", "Neu_2024"
]

print("✓ Data loaded and cleaned\n")

# =====
# Company Waterfall: sentiment deterioration
# =====

def create_company_waterfall():
    """
    Waterfall-style chart showing how company sentiment
    changed across key credit risk sections.
    """

    key_sections = [
        "Credit Highlights: Key risks",
        "Executive Summary: Risk",
        "Executive Summary: Debt/leverage risk",
        "Outlook",
        "Business Risk",
        "Financial Risk",
        "Liquidity: Overview",
    ]

```

```
df = company[company["Section"].isin(key_sections)].copy()
```

```
df["Net_2023"] = df["Pos_2023"] - df["Neg_Val_2023"]
```

```
df["Net_2024"] = df["Pos_2024"] - df["Neg_Val_2024"]
```

```
df["Change"] = df["Net_2024"] - df["Net_2023"]
```

```
df = df.sort_values("Change")
```

```
labels = [s.split(":")[-1].strip() for s in df["Section"]]
```

```
x = np.arange(len(labels))
```

```
fig, ax = plt.subplots(figsize=(14, 7))
```

```
colors = [
```

```
    COLORS["risk_high"] if c < 0 else COLORS["risk_low"]
```

```
    for c in df["Change"]
```

```
]
```

```
bars = ax.bar(x, df["Change"], color=colors, alpha=0.85,
```

```
              edgecolor="black", linewidth=1.2)
```

```
ax.axhline(0, color="black", linewidth=1.5)
```

```
for bar, val in zip(bars, df["Change"]):
```

```
    y = val + 0.01 if val > 0 else val - 0.02
```

```
    ax.text(bar.get_x() + bar.get_width() / 2,
```

```
            y, f"{val:.3f}",
```

```
            ha="center",
```

```
            va="bottom" if val > 0 else "top",
```

```
            fontsize=9, fontweight="bold")
```

```
ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=45, ha="right")
ax.set_ylabel("Change in Net Sentiment (2023 → 2024)")
ax.set_title("Company Credit Risk: Sentiment Deterioration by Section")
```

```
plt.tight_layout()
plt.savefig("company_waterfall_sentiment.png", dpi=300)
plt.close()
```

```
print("✓ company_waterfall_sentiment.png")
```

```
# =====
# Company Heatmap: section-wise negativity
# =====
```

```
def create_company_heatmap():
```

```
    """
```

```
    Heatmap of negative sentiment intensity across sections.
```

```
    """
```

```
    df = company[company["Section"].notna()].copy()
```

```
    sections = df["Section"].values[:15]
```

```
    labels = [s[:35] + "..." if len(s) > 35 else s for s in sections]
```

```
    data = np.column_stack([
        df["Neg_Val_2023"].values[:15],
        df["Neg_Val_2024"].values[:15],
    ])
```

```
    fig, ax = plt.subplots(figsize=(10, 10))
```

```
    im = ax.imshow(data, cmap="YlOrRd", vmin=0, vmax=1)
```

```

ax.set_xticks([0, 1])
ax.set_xticklabels(["2023", "2024"], fontweight="bold")
ax.set_yticks(range(len(labels)))
ax.set_yticklabels(labels)

for i in range(data.shape[0]):
    for j in range(data.shape[1]):
        ax.text(j, i, f"{data[i, j]:.2f}",
                ha="center", va="center",
                fontsize=9,
                color="white" if data[i, j] > 0.5 else "black")

cbar = plt.colorbar(im, ax=ax)
cbar.set_label("Negativity Rate", rotation=270, labelpad=20)

ax.set_title("Company Credit Risk Heatmap")

plt.tight_layout()
plt.savefig("company_risk_heatmap.png", dpi=300)
plt.close()

print("✓ company_risk_heatmap.png")

```

```

# =====
# Company Sentiment Gauge (executive summary)
# =====

```

```

def create_company_summary():
    """
    High-level comparison of overall sentiment composition
    and net sentiment trend.
    """

```

```
avg_2023 = company[["Pos_2023", "Neg_Val_2023", "Neu_2023"]].mean()
avg_2024 = company[["Pos_2024", "Neg_Val_2024", "Neu_2024"]].mean()
```

```
net_2023 = avg_2023["Pos_2023"] - avg_2023["Neg_Val_2023"]
net_2024 = avg_2024["Pos_2024"] - avg_2024["Neg_Val_2024"]
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
```

```
years = ["2023", "2024"]
pos = [avg_2023["Pos_2023"], avg_2024["Pos_2024"]]
neg = [avg_2023["Neg_Val_2023"], avg_2024["Neg_Val_2024"]]
neu = [avg_2023["Neu_2023"], avg_2024["Neu_2024"]]
```

```
x = np.arange(len(years))
ax1.bar(x, pos, color=COLORS["positive"])
ax1.bar(x, neg, bottom=pos, color=COLORS["negative"])
ax1.bar(x, neu, bottom=np.array(pos) + np.array(neg),
        color=COLORS["neutral"])
```

```
ax1.set_xticks(x)
ax1.set_xticklabels(years)
ax1.set_ylim(0, 1)
ax1.set_title("Sentiment Composition")
```

```
ax2.bar(years, [net_2023, net_2024],
        color=[COLORS["risk_low"], COLORS["risk_high"]])
ax2.axhline(0, color="black", linestyle="--")
ax2.set_title("Net Sentiment Trend")
```

```
plt.tight_layout()
plt.savefig("company_sentiment_gauge.png", dpi=300)
```

```
plt.close()
```

```
print("✓ company_sentiment_gauge.png")
```

```
# =====  
# Industry visualisations (trend + risk distribution)  
# =====
```

```
def create_industry_overview():
```

```
    """
```

```
    Industry-wide comparison of positive vs negative sentiment.
```

```
    """
```

```
    key_sections = [
```

```
        "What's changed?",
```

```
        "key risks",
```

```
        "Industry Outlook: Ratings trends and outlook",
```

```
        "Main assumptions about 2024 and beyond",
```

```
        "Key risks or opportunities around the baseline",
```

```
    ]
```

```
    df = industry[industry["Section"].isin(key_sections)]
```

```
    x = np.arange(len(df))
```

```
    width = 0.35
```

```
    fig, ax = plt.subplots(figsize=(14, 7))
```

```
    ax.bar(x - width / 2, df["Pos_2024"], width,
```

```
          label="Positive", color=COLORS["positive"])
```

```
    ax.bar(x + width / 2, df["Neg_Val_2024"], width,
```

```
          label="Negative", color=COLORS["negative"])
```



```
ax.set_xticks(x)
ax.set_xticklabels(df["Section"], rotation=45, ha="right")
ax.set_title("Industry Credit Outlook")
ax.legend()
```

```
plt.tight_layout()
plt.savefig("industry_sentiment_evolution.png", dpi=300)
plt.close()
```

```
print("✓ industry_sentiment_evolution.png")
```

```
# =====
```

```
# Run all plots
```

```
# =====
```

```
if __name__ == "__main__":
```

```
    print("\nGenerating company figures...")
```

```
    create_company_waterfall()
```

```
    create_company_heatmap()
```

```
    create_company_summary()
```

```
    print("\nGenerating industry figures...")
```

```
    create_industry_overview()
```

```
    print("\nAll figures generated successfully.")
```

```
.....
```

Cluster-Level Sentiment Analysis (K-Means Themes)

The focus here is to create diagrams for cluster level analysis (which perhaps no one will see :()

.....

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec
```

```
# -----
```

```
# Styling
```

```
# -----
```

```
plt.style.use("seaborn-v0_8-darkgrid")
plt.rcParams["figure.dpi"] = 300
plt.rcParams["savefig.dpi"] = 300
```

```
COLORS = {
    "company_2023": "#3498db",
    "company_2024": "#e74c3c",
    "industry_2024": "#2ecc71",
    "industry_2025": "#9b59b6",
    "positive": "#27ae60",
    "negative": "#c0392b",
    "neutral": "#95a5a6",
    "accent": "#f39c12",
}
```

```
# =====
```

```
# Load cluster-level sentiment data
```

```
# =====
```

```
print("Loading cluster sentiment data...")
```

```

cluster_df = pd.read_excel("final_cluster_sentiment.xlsx", sheet_name=0)

print(f"✓ {len(cluster_df)} observations loaded")

# =====
# Cluster landscape heatmap
# =====

def cluster_heatmap():
    """
    Heatmap of net sentiment across clusters, entities, and years.
    """

    pivot = cluster_df.pivot_table(
        values="net_sentiment",
        index="cluster_name",
        columns=["entity", "year"],
        aggfunc="mean",
    )

    pivot["avg"] = pivot.mean(axis=1)
    pivot = pivot.sort_values("avg").drop(columns="avg")

    fig, ax = plt.subplots(figsize=(16, 10))
    im = ax.imshow(pivot.values, cmap="RdYlGn",
                   vmin=-0.5, vmax=0.2)

    ax.set_yticks(range(len(pivot.index)))
    ax.set_yticklabels(pivot.index, fontsize=9)

    ax.set_xticks(range(len(pivot.columns)))
    ax.set_xticklabels(

```

```

        [f"{c[0]}\n{c[1]}" for c in pivot.columns],
        fontweight="bold"
    )

    for i in range(pivot.shape[0]):
        for j in range(pivot.shape[1]):
            val = pivot.iloc[i, j]
            if not np.isnan(val):
                ax.text(j, i, f"{val:.2f}",
                        ha="center", va="center",
                        fontsize=8,
                        color="white" if abs(val) > 0.15 else "black")

    plt.colorbar(im, ax=ax, label="Net Sentiment")
    ax.set_title("Cluster Risk Landscape")

    plt.tight_layout()
    plt.savefig("cluster_landscape_heatmap.png", dpi=300)
    plt.close()

    print("✓ cluster_landscape_heatmap.png")

# =====

# Run

# =====

if __name__ == "__main__":
    cluster_heatmap()
    print("Cluster visualisations complete.")

```